

Sistema de Deteccion de Armas mediante Vision Artificial con YOLOv8

Documento tecnico final del proceso completo

Plataforma de entrenamiento: NVIDIA RTX 5050 Laptop (Blackwell)

Plataforma de despliegue: Raspberry Pi 5

Fecha: 07/05/2026 15:56

Repositorio: github.com/Enryuuu/deteccion-de-armas-i.a

Indice

1. Resumen ejecutivo
2. Objetivos del sistema
3. Arquitectura de hardware
4. Stack de software
5. Datasets y procesamiento de datos
6. Modelo y arquitectura YOLO
7. Configuración del entrenamiento
8. Fases ejecutadas (cronología completa)
9. Resultados finales
10. Cuantización INT8 para Raspberry Pi
11. Despliegue en Raspberry Pi 5
12. Guía: SSH a la Pi desde Visual Studio Code
13. Riesgos identificados y mitigaciones
14. Reproducibilidad y trabajo futuro

1. Resumen ejecutivo

El presente documento describe el desarrollo completo de un sistema de detección en tiempo real de armas blancas (cuchillos) y armas de fuego (pistolas, escopetas, rifles) basado en la arquitectura YOLOv8 de Ultralytics. El sistema fue entrenado en una laptop ASUS TUF Gaming A16 con GPU NVIDIA RTX 5050 (Blackwell, sm_120) y exportado a ONNX cuantizado INT8 para despliegue en Raspberry Pi 5.

Se ejecutaron **dos iteraciones** del proceso de entrenamiento. La versión v1 utilizó únicamente Open Images v7 con dos clases agregadas (knife/firearm) y obtuvo un mAP@0.5 de 0.62 en test. La versión v2 incorporó datasets de Roboflow Universe, elevó el corpus a 8.656 imágenes curadas y descompuso la clase firearm en **handgun** y **long_gun**, alcanzando un **mAP@0.5 de 0.73 en test** con la clase handgun pasando de 0.49 a 0.80.

Adicionalmente se entrenó un **YOLOv8n** con el mismo dataset para despliegue en la Pi, obteniendo un mAP@0.5 de 0.83 en val y un tamaño final cuantizado a INT8 de **3.25 MB** (72% menor que el FP32). El modelo final está listo para inferir en tiempo real en la Raspberry Pi 5.

2. Objetivos del sistema

- Detectar en tiempo real armas blancas (clase **knife**) y armas de fuego diferenciando armas cortas (**handgun**) de armas largas (**long_gun**: rifles, escopetas).
- Lograr una velocidad de inferencia mínima de 10 FPS en Raspberry Pi 5 a resolución 416x416.
- Mantener el modelo desplegado en menos de 5 MB para permitir actualizaciones remotas eficientes (objetivo cumplido: 3.25 MB).
- Disparar alertas locales (sonido, log y captura de evidencia) cuando se detecte una amenaza con confianza superior al umbral configurado.
- Permitir el despliegue completo en la Pi sin necesidad de instalar PyTorch ni Ultralytics, reduciendo la huella a menos de 250 MB de paquetes Python.

3. Arquitectura de hardware

3.1 Estacion de entrenamiento

Componente	Especificacion	Rol
Equipo	ASUS TUF Gaming A16 FA608UH	Entrenamiento + inferencia desktop
CPU	AMD Ryzen 7 260 (8 cores / 16 hilos)	DataLoader workers
RAM	32 GB DDR5	Cache de dataset
GPU dedicada	NVIDIA RTX 5050 Laptop, 8 GB VRAM	Entrenamiento CUDA
Arquitectura GPU	Blackwell, capability sm_120	Requiere CUDA 12.8+
GPU integrada	AMD Radeon 780M	Solo display, sin CUDA
Driver NVIDIA	581.80 (CUDA 13.0 runtime)	Verificado al iniciar

3.2 Plataforma de despliegue

Componente	Recomendado
SBC	Raspberry Pi 5 (8 GB RAM)
Almacenamiento	microSD 64 GB clase A2
Refrigeracion	Active cooler oficial (imprescindible bajo carga)
Camara	USB UVC o Pi Camera v3
Sistema operativo	Raspberry Pi OS Bookworm 64-bit
Alimentacion	Fuente oficial 5V/5A

4. Stack de software

El entorno se aísla en un virtualenv Python 3.11.9 dentro del repositorio (repo/.venv). Se forzó la asignación de la GPU NVIDIA para los ejecutables Python vía la clave de registro HKCU\Software\Microsoft\DirectX\UserGpuPreferences.

Paquete	Version	Funcion
torch	2.11.0+cu128	Framework de deep learning con soporte CUDA 12.8
torchvision	0.26.0+cu128	Operaciones de vision en PyTorch
ultralytics	8.4.47	Framework YOLOv8 (entrenamiento + export)
fiftyone	1.15.0	Gestion del dataset Open Images v7
roboflow	1.x	Descarga de datasets Roboflow Universe
opencv-python	4.13.0	Captura de camara y dibujo de bboxes
onnx + onnxruntime	1.21 / 1.25	Export e inferencia portable + cuantizacion
onnxslim	0.1.92	Optimizacion del grafo ONNX
pygame	2.6.1	Reproduccion de alertas sonoras
reportlab	4.5.0	Generacion del presente documento

5. Datasets y procesamiento de datos

5.1 Fuentes utilizadas

Fuente	Imagenes	Clases originales	Mapeo final
Open Images v7 (FiftyOne)	2.185	Knife, Handgun, Shotgun, Rifle	knife / handgun / long_gun
RF joseph-nelson/pistols	2.971	pistol	handgun
RF rifledetection/rifle-detection-p9slr	3.500*	Rifle	long_gun

* *Sampleadas aleatoriamente desde un total de 13.013 imagenes para mantener balance de clases.*

5.2 Reorganizacion taxonomica (decision clave del v2)

La version v1 utilizaba una clase generica **firearm** que agrupaba pistolas, escopetas y rifles. Esta decision degrado el rendimiento en esa clase a un mAP@0.5 de 0.49 debido a la alta varianza visual entre los subtipos. Para v2 se establecio la siguiente taxonomia final:

- **0 — knife:** cuchillos y armas blancas en general.
- **1 — handgun:** armas cortas (pistolas, revolveres). Empuniadura a una mano.
- **2 — long_gun:** armas largas (rifles, escopetas). Requieren dos manos.

5.3 Particion final del dataset v2

Particion	Imagenes	Anotaciones knife	Anotaciones handgun	Anotaciones long_gun
Train (85%)	7.357	983	3.593	5.363
Val (10%)	865	93	413	699
Test (5%)	434	51	217	322
Total	8.656	1.127	4.223	6.384

5.4 Formato YOLO

Cada imagen NNNNNNN.jpg tiene asociado un archivo NNNNNNN.txt con una linea por objeto detectado:

```
<class_id> <cx> <cy> <ancho> <alto>
```

Donde todas las coordenadas estan normalizadas en el rango [0, 1] respecto al tamaño de la imagen, y (cx, cy) es el centro del bounding box.

6. Modelo y arquitectura YOLO

YOLO (You Only Look Once) es una familia de detectores de objetos en una sola pasada. A diferencia de R-CNN que primero propone regiones y luego las clasifica, YOLO predice simultaneamente clase y bounding box procesando la imagen una unica vez, lo que la convierte en la opcion adecuada para inferencia en tiempo real en hardware limitado como la Raspberry Pi.

6.1 Variantes evaluadas

Variante	Parametros	Tamano FP32	Uso en este proyecto
YOLOv8n (nano)	3.2 M	~6 MB / 12 MB ONNX	Despliegue final en Pi5
YOLOv8s (small)	11 M	~22 MB	Entrenamiento en laptop (referencia)
YOLOv8m (medium)	26 M	~52 MB	No utilizado

6.2 Estrategia dual de modelos

Se entrenaron **dos modelos en paralelo** sobre el mismo dataset:

- **YOLOv8s** (small) como modelo de referencia con la mejor precision posible. Permite validar la calidad del dataset y servir como benchmark.
- **YOLOv8n** (nano) optimizado para tamano y velocidad. Es el modelo que se exporta a ONNX cuantizado INT8 para correr en la Pi.

El nano alcanzo un mAP@0.5 muy similar al small (0.83 vs 0.84) pero con **3.6x menos peso** y **4.4x mas velocidad**, validando la decision.

7. Configuracion del entrenamiento

7.1 Hiperparametros YOLOv8s v2

Parametro	Valor	Justificacion
epochs	100	Suficiente con dataset ampliado
batch	8	Reducido desde 12 (v1 sufrio OOM en epoch 70)
workers	4	Reducido desde 8 para liberar RAM
imgsz	640	Estandar YOLOv8 en GPU
optimizer	AdamW	Adaptativo, robusto, estandar moderno
lr0	0.001	Conservador para fine-tuning
patience	30	Early stopping si val no mejora
amp	true	FP16 mixed precision (mitad de VRAM)
device	0	GPU CUDA 0 (RTX 5050)

7.2 Hiperparametros YOLOv8n

Parametro	Valor	Justificacion
model	yolov8n.pt	Pesos preentrenados en COCO
epochs	80	Modelo mas pequeno converge antes
batch	24	32 dispara cuDNN error en Blackwell, 24 estable
workers	6	Aprovecha mejor las 16 hilos del CPU
imgsz	416	Resolucion reducida para FPS en Pi
cache	ram	8.6k imgs en RAM (~3 GB) -> sin lectura disco

7.3 Augmentations aplicadas

Augmentation	Valor	Efecto
flipplr	0.5	50% prob. espejo horizontal
flipud	0.0	Sin espejo vertical (irreal)
mosaic	1.0	Combina 4 imagenes en una
mixup	0.15 (v8s) / 0.1 (v8n)	Transparenta dos imagenes
copy_paste	0.3	Copia objetos entre imagenes (clave para knife)
hsv_h/s/v	0.015 / 0.7 / 0.4	Variacion de tono, saturacion, brillo

7.4 Variables de entorno de seguridad

```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
CUDA_VISIBLE_DEVICES=0
KMP_DUPLICATE_LIB_OK=TRUE
```

`expandable_segments:True` es la mitigacion clave del incidente de OOM observado en la version v1 (epoch 70/80). Permite a PyTorch expandir segmentos de VRAM existentes en lugar de fragmentar la memoria con asignaciones nuevas.

8. Fases ejecutadas (cronologia completa)

Fase	Nombre	Descripcion	Estado
Fase 0	Setup del entorno	Instalacion de Python 3.11.9, virtualenv, PyTorch 2.11+cu128 (verificado torch.cuda.is_available()=True, RTX 5050 sm_120), instalacion del requirements.txt. Configuracion de preferencia de GPU NVIDIA en el registro de Windows.	Completado
Fase 1	Descarga inicial de Open Images v7 (61)	Primer ejecucion de 1_download_dataset.py. Se identificaron dos bugs: (1) la clase 'Firearm' no existe como leaf class en Olv7; (2) el parametro dataset_dir entraba en conflicto con argumentos internos de FiftyOne. Ambos corregidos, descarga exitosa de 2.185 imagenes en 4 clases.	Completado
Fase 2	Conversion a formato YOLOv8	Mapeo Olv7 -> 2 clases (knife/firearm), generacion de splits y data.yaml. 1.500 train + 169 val + 516 test = 2.185 imagenes con 3.612 anotaciones validas.	Completado
Fase 3	Entrenamiento YOLOv8s	100 epochs configuradas, batch 12, workers 8. Crash con CUDA OOM en epoch 70/80 por fragmentacion de VRAM. best.pt se conservo.	Completado con incidente
Fase 4	Evaluacion v1	Sobre el split de test (516 imgs): mAP@0.5 = 0.617, mAP@0.5:0.95 = 0.474, Precision = 0.83, Recall = 0.65. Velocidad: 4.4 ms/img (227 FPS en RTX). Identificado bajo rendimiento en clase firearm (0.49).	Completado
Fase 5	Plan de mejora y obtencion de dataset	Estudio de Open Images v7 corpus con datasets curados de RF Universe, descomponer firearm en handgun + long_gun, ajustar config para evitar OOM, planificar export INT8.	Completado
Fase 6	Descarga de datasets	Robert-nelson/pistols (2.971 imgs) y rifledetection/rifle-detection-p9slr (13.013 imgs, sampleadas a 3.500). Total descargado: ~1.8 GB.	Completado
Fase 7	Merge y unificacion del dataset	Implementacion de 2b_merge_datasets.py: re-mapeo a 3 categorias, particion estratificada 85/10/5, validacion de imagenes corruptas. Resultado: 8.656 imagenes finales.	Completado
Fase 8	Reentrenamiento YOLOv8s	200 epochs en 2h 55min. Sin OOM. mAP@0.5 val final: 0.838. Best epoch ~80 con metrica estable.	Completado
Fase 9	Evaluacion v2 en test	mAP@0.5 = 0.729 (+11pp vs v1). Por clase: knife 0.72, handgun 0.80, long_gun 0.66.	Completado

Fase 10	Entrenamiento YOLOv8n en Pi	90 epochs en 52 min con cache RAM. mAP@0.5 val: 0.833 (-0.5pp vs YOLOv8s). Modelo final: 6.2 MB.	Completado
Fase 11	Export ONNX + cuantización	ONNX FP32: 11.6 MB. ONNX INT8 calibrado con 100 imgs: 3.25 MB (-72%). Listo para copiar a la Pi.	Completado
Fase 12	Publicacion en GitHub	Commit con todos los cambios: scripts modificados, scripts nuevos (2b, 3b, 6b), configuracion v2, documentacion. Push exitoso a master.	Completado

9. Resultados finales

9.1 Comparativa global v1 vs v2

Metrica	v1 (Olv7, 2 clases)	v2 (Olv7+RF, 3 clases)	Delta
Imagenes totales	2.185	8.656	+296%
mAP@0.5 (test)	0.617	0.729	+11.2 pp
mAP@0.5:0.95 (test)	0.474	0.589	+11.5 pp
Precision	0.83	0.87	+4 pp
Recall	0.65	0.74	+9 pp

9.2 Resultados v2 por clase (split test)

Clase	Instancias	Precision	Recall	mAP@0.5	mAP@0.5:0.95
all	589	0.868	0.745	0.729	0.589
knife	51	0.760	0.745	0.724	0.655
handgun	217	0.951	0.806	0.802	0.649
long_gun	321	0.894	0.682	0.660	0.464

Hallazgo clave: la clase handgun paso de 0.49 (v1, agregada como firearm) a 0.80 (v2, separada). La descomposicion taxonomica fue la mejora individual mas grande.

9.3 YOLOv8n vs YOLOv8s (val)

Metrica	YOLOv8s (small)	YOLOv8n (nano)	Diferencia
mAP@0.5 global	0.838	0.833	-0.5%
mAP@0.5 knife	0.858	0.845	-1.3%
mAP@0.5 handgun	0.908	0.883	-2.5%
mAP@0.5 long_gun	0.748	0.772	+2.4%
Tamano (.pt)	22.5 MB	6.2 MB	-72%
Inferencia	3.1 ms/img	0.7 ms/img	-77%
FPS en RTX	~322	~1428	+343%

9.4 Curva de aprendizaje YOLOv8s v2

Epoch	mAP@0.5	mAP@0.5:0.95
1	0.443	0.246
12	0.681	0.431
26	0.780	0.549
47	0.820	0.598
71	0.844	0.625
100 (final)	0.838	0.639

10. Cuantizacion INT8 para Raspberry Pi

La cuantizacion convierte los pesos del modelo de 32 bits flotantes (FP32) a 8 bits enteros (INT8). Esto reduce el tamaño del archivo en ~75% y acelera la inferencia en hardware ARM en aproximadamente 4x, con una pérdida típica de mAP menor al 2%.

10.1 Proceso aplicado

Se utilizó `onnxruntime.quantization.quantize_static()` con calibración estática:

- Modelo de entrada: `yolov8n_v2_fp32.onnx` (11.6 MB)
- Set de calibración: 100 imágenes seleccionadas aleatoriamente del split de train
- Formato: QDQ (Quantize-DeQuantize), per-channel
- Pesos: QInt8 | Activaciones: QUInt8
- Método de calibración: MinMax

10.2 Resultado

Modelo	Formato	Tamaño	Reducción
<code>yolov8n_v2_fp32.onnx</code>	FP32	11.61 MB	baseline
<code>yolov8n_v2_int8.onnx</code>	INT8 estático QDQ	3.25 MB	-72%

11. Despliegue en Raspberry Pi 5

11.1 Software a instalar en la Pi

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-pip python3-venv libatlas-base-dev libopenblas-dev
python3 -m venv ~/weapons-env
source ~/weapons-env/bin/activate
pip install opencv-python onnxruntime numpy pyyaml pygame
```

Total instalado: ~250 MB. **No** se instala PyTorch ni Ultralytics.

11.2 Archivos a copiar desde la laptop

Archivo	Función	Tamaño
<code>models/export/yolov8n_v2_int8.onnx</code>	Modelo cuantizado INT8	3.25 MB
<code>config.yaml</code>	Umbral y configuración	2 KB
<code>utils/</code>	Visualización + alertas	10 KB
<code>scripts/7_inference_pi.py</code>	Loop de inferencia	5 KB

11.3 Comando scp para transferir

```
scp models/export/yolov8n_v2_int8.onnx pi@<ip-pi>:~/weapons/best.onnx
scp config.yaml pi@<ip-pi>:~/weapons/
scp -r utils pi@<ip-pi>:~/weapons/
scp scripts/7_inference_pi.py pi@<ip-pi>:~/weapons/
```

11.4 Performance esperado

Resolucion	Modelo	FPS esperados en Pi 5
416x416	YOLOv8n FP32 ONNX	4 - 7
416x416	YOLOv8n INT8 ONNX	12 - 18 (objetivo)
320x320	YOLOv8n INT8 ONNX	18 - 25
416x416	YOLOv8n NCNN FP16	15 - 22

12. Guia: SSH a la Pi desde Visual Studio Code

Esta seccion documenta el flujo completo para administrar y desarrollar en la Raspberry Pi 5 desde Visual Studio Code en la laptop, usando la extension Remote-SSH.

12.1 Setup inicial de la Pi

- Flashear microSD con **Raspberry Pi Imager** (Pi OS Lite 64-bit).
- En el menu avanzado del Imager (icono de engranaje), configurar:
 - - Hostname: raspberrypi
 - - Habilitar SSH (con autenticacion por password o llave)
 - - Usuario y password (ej. pi / tu password)
 - - Configurar WiFi (SSID + password)
- Insertar la SD en la Pi, conectar fuente y esperar 60-90 segundos al primer boot.

12.2 Encontrar la IP de la Pi

Desde la laptop, opcion mas rapida (ping mDNS):

```
ping raspberrypi.local
```

Si no responde, escanear la red:

```
# Windows (PowerShell)
arp -a | findstr 'b8-27-eb\|dc-a6-32\|e4-5f-01\|2c-cf-67'

# Linux/Mac
nmap -sn 192.168.1.0/24
```

12.3 Configurar SSH key (recomendado)

Desde la laptop, generar par de llaves si no existe:

```
ssh-keygen -t ed25519 -C 'pi-deploy'
```

Copiar la llave publica a la Pi:

```
# Windows PowerShell
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh pi@<ip> 'cat &&gt;&>
~/.ssh/authorized_keys'

# Linux/Mac
ssh-copy-id pi@<ip>
```

A partir de aqui no se pide password al conectar.

12.4 Editar ~/.ssh/config en la laptop

Esto permite conectarse simplemente con ssh pi5:

```
Host pi5
HostName 192.168.1.42 # reemplazar por la IP real
User pi
IdentityFile ~/.ssh/id_ed25519
ServerAliveInterval 60
```

12.5 Instalar la extension Remote-SSH en VS Code

- Abrir Visual Studio Code en la laptop.
- Ir a Extensions (Ctrl+Shift+X).
- Buscar e instalar: **Remote - SSH** (publisher: ms-vscode-remote).
- Opcionalmente instalar tambien **Remote - SSH: Editing Configuration Files**.

12.6 Conectarse a la Pi desde VS Code

- Abrir Command Palette: Ctrl+Shift+P.
- Escribir: **Remote-SSH: Connect to Host...**
- Seleccionar pi5 (o el alias configurado).
- Se abre una nueva ventana de VS Code conectada a la Pi.
- La primera vez descarga el VS Code Server en la Pi (~80 MB, una sola vez).
- Esquina inferior izquierda mostrara: SSH: pi5 (verde).

12.7 Estructura de carpetas en la Pi

```
/home/pi/  
weapons-env/ # virtualenv  
weapons/  
best.onnx # modelo INT8 (3.25 MB)  
config.yaml  
utils/  
visualization.py  
alerts.py  
7_inference_pi.py  
logs/  
detections.log  
frames/ # capturas de evidencia
```

12.8 Subir archivos: dos opciones

Opcion A — arrastrar y soltar en VS Code:

- Una vez conectado, abrir el explorador de archivos (Ctrl+Shift+E).
- Navegar a /home/pi/weapons.
- Arrastrar archivos desde el explorador del SO directamente a la ventana.

Opcion B — via terminal con scp:

```
scp models/export/yolov8n_v2_int8.onnx pi5:~/weapons/best.onnx  
scp config.yaml pi5:~/weapons/  
scp -r utils pi5:~/weapons/  
scp scripts/7_inference_pi.py pi5:~/weapons/
```

12.9 Ejecutar inferencia desde la terminal de VS Code

- En VS Code conectado a la Pi, abrir terminal: Ctrl+\`.
- Activar el venv:

```
source ~/weapons-env/bin/activate
```

- Ejecutar inferencia:

```
cd ~/weapons && python 7_inference_pi.py --weights best.onnx
```

- Para detener: Ctrl+C.

12.10 Debugging remoto con breakpoints

En la pestaña **Run and Debug** (Ctrl+Shift+D), crear `.vscode/launch.json` en la Pi con:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Inferencia Pi",
      "type": "debugpy",
      "request": "launch",
      "program": "${workspaceFolder}/7_inference_pi.py",
      "args": ["--weights", "best.onnx"],
      "console": "integratedTerminal",
      "python": "/home/pi/weapons-env/bin/python"
    }
  ]
}
```

Ahora F5 lanza el script con breakpoints funcionales.

12.11 Servicio systemd para arranque automatico

Para que la detección arranque sola cada vez que se enchufa la Pi, crear `/etc/systemd/system/weapons.service`:

```
[Unit]
Description=Weapons Detection
After=network.target

[Service]
User=pi
WorkingDirectory=/home/pi/weapons
ExecStart=/home/pi/weapons-env/bin/python 7_inference_pi.py --weights best.onnx
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Activar:

```
sudo systemctl daemon-reload
sudo systemctl enable weapons
sudo systemctl start weapons
sudo systemctl status weapons # verificar
```


12.12 Comandos utiles desde la terminal

Comando	Funcion
journalctl -u weapons -f	Ver logs en vivo del servicio
sudo systemctl restart weapons	Reiniciar deteccion
htop	Monitor de CPU y RAM
vcgencmd measure_temp	Temperatura del SoC
v4l2-ctl --list-devices	Listar camaras detectadas
uptime	Tiempo encendido + carga
df -h /	Espacio en disco

12.13 Troubleshooting comun

Problema	Solucion
Camara no detectada (cv2.VideoCapture(0))	Probar otros indices (1, 2). Verificar permisos: sudo usermod -aG video pi
FPS muy bajos (<5)	Verificar throttling: vcgencmd get_throttled. Si distinto de 0x0, problema termico/voltaje
No se reproduce sonido	Verificar audio: aplay -l. Configurar default device en ~/.asoundrc
VS Code Remote-SSH no conecta	Borrar ~/.vscode-server en la Pi y reintentar. Verificar version de SSH (>=8.0)
systemctl status reporta failed	Ver journalctl -u weapons -n 50 para detalle
Poca memoria al inferir	Cerrar X11/desktop si se uso Pi OS Desktop (preferir Lite)

13. Riesgos identificados y mitigaciones

Riesgo	Impacto	Mitigacion aplicada
OOM de VRAM al final del entrenamiento	Crash en epoch 70/80	batch 12->8, workers 8->4, expandable_segments:True
Fragmentacion progresiva de VRAM	OOM pre-epoch	PYTORCH_CUDA_ALLOC_CONF configurada
Clase minoritaria knife (12% del dataset)	Recall en cuchillos	copy_paste augmentation 0.3
TDR de Windows durante carga GPU	Pantalla negra momentanea	Inocuo: el computo CUDA no se interrumpe
GPU NVIDIA Blackwell sm_120 incompatible con CUDA 12.9	Crash al cargar PyTorch	PyTorch instalado contra cu128 explicitamente
batch 32 dispara cuDNN error en yolo8n	Crash en epoch 1	Reducido a batch 24
Falta de assets/alert.wav en el repo	Crash al activar alertas	alerts.sound_enabled=false hasta proveer el archivo
Pistola 3D impresa fuera de distribucion	Bajo confidence en pruebas	Imprimir en negro mate; bajar threshold a 0.30 si necesario

14. Reproducibilidad y trabajo futuro

14.1 Para reproducir todo desde cero

Ver el archivo README . txt en la raiz del repositorio.

14.2 Trabajo futuro sugerido

- Incorporar 200-500 hard negatives (utensilios de cocina, controles, taladros) para reducir falsos positivos.
- Ampliar dataset con mas ejemplos de armas en contextos reales (manos, ropa, interiores, exteriores).
- Probar variantes YOLOv8 mas grandes (m, l) en la laptop para tener un modelo "teacher" para destilacion al nano.
- Generar archivo de alerta sonora assets/alert . wav.
- Implementar deteccion de tracking entre frames (ByteTrack/BoTSORT) para reducir alertas duplicadas y mejorar la confiabilidad temporal.
- Integrar con un sistema de notificacion remota (Telegram, email) para alertas en ubicaciones desatendidas.
- Optimizar el modelo NCNN para ARM y comparar FPS reales contra ONNX Runtime.
- Considerar deteccion en stream RTSP (camaras IP) ademas de USB/Pi Camera.